LuminaX Round 2 final report

Stable reciprocal matching with observable-only evidence

Item	Submission detail
Team	LuminaX
Members	Ahan Sardar and Enakshee Mondal
Submitted method	`adaptive_greedy`
Policy entry point	`policy.py`
Runtime implementation	`adaptive.py`
Result summary	`results/round2_summary.json`
Prepared	14 October 2026

1. What we submit

We submit a deterministic, offline policy for the two-phase JSON interface. It asks for unresolved hard constraints, rebuilds the feasible pair graph after clarification, scores only feasible pairs, and returns a non-overlapping daily matching. The Docker entry point uses `adaptive_greedy` without a command-line flag.

The policy does not use a trained model. It does not report probabilities. It does not use network access, hidden simulator state, generator seeds, or member IDs as outcome features. Opaque member IDs appear only in canonical sorting so the same observable state gives the same action.

Our main claim is narrow. Stable clarification order is a correctness fix that also produced two more MSMI outcomes than the supplied greedy policy in the declared 18-episode public matrix. The sample is small, so this result does not establish a private-world advantage.

2. Problem interpretation

This is sequential allocation under uncertainty, not independent pair classification. A valid action must satisfy four conditions:

1. Both people satisfy every reciprocal hard constraint.
2. Missing hard information blocks the introduction until it is clarified.
3. No person appears twice in a batch or while another introduction is open.
4. The policy must choose across the full daily pool because one match removes two people from every competing edge.

The official objective is the mean MSMI per 100 arrived members across six equally weighted scenario families. Coverage, mutual acceptance, clarification cost, and runtime are tie-breakers in that order. We use that order throughout the report.

3. Daily policy

3.1 Validate the observable state

The policy accepts one row per opaque member ID. It rejects malformed rows, future observations, inconsistent missing-value states, and conflicting duplicate rows. This fail-closed rule prevents input order from deciding which version of a person the policy trusts.

3.2 Ask only for blocking constraints

The policy finds available people with at least one missing hard field. It skips anyone who declined a hard field because the simulator will not reveal a declined answer on another request. It then sorts candidates by arrival day and opaque ID.

One hard-constraint bundle costs 3 units. With budget `B`, the policy asks at most `floor(B / 3)` people. It never spends the clarification budget on soft fields because the submitted method has no validated evidence that those asks improve the official outcome.

3.3 Build the reciprocal feasible graph

After clarification, the evaluator supplies a refreshed observation. The policy keeps only available people with complete hard constraints. It adds an edge only if the official eligibility function confirms reciprocal age and stated gender preferences, compatible relationship structure, smoking and children constraints, mutually accepted current zones, at least one shared schedule slot, no repeated pair, and no open introduction for either person.

A score cannot override one of these checks.

3.4 Score feasible pairs

For each feasible pair, the score is the number of observed soft fields with equal values. Missing soft fields add zero. They are not treated as mismatch, rejection, or evidence of low compatibility.

This score is an allocation priority, not a calibrated relationship probability. We do not label it as predicted acceptance or predicted MSMI.

3.5 Allocate the batch

The policy sorts edges by descending soft-field score and then by canonical pair ID. It accepts an edge when neither endpoint has already been used in the batch. The result is deterministic and non-overlapping.

The selected allocator is greedy. We tested maximum-cardinality, history-based, learned, waiting-priority, and graph-aware variants. Their gains did not repeat on untouched seed blocks, or they lost an official tie-break. The rejected methods remain documented in `docs/DECISIONS.md`, but the container does not call them.

4. Missing and delayed information

The policy treats each type of uncertainty separately:

Situation	Policy behavior
Hard field is not asked	Request the hard bundle when budget and priority allow.
Hard field was declined	Do not ask again. Keep unresolved edges blocked.
Soft field is missing	Give no score credit and no mismatch penalty.
Introduction response is pending	Respect the open-introduction block. Do not infer rejection.
Feedback arrives late	Use only events present in the current request. The selected method does not score from history.
Member becomes unavailable	Exclude the member before asking and matching.
No feasible edge exists	Return an empty pair list.

The selected method does not use delayed feedback as a predictive signal. Earlier history-based variants had uncertain or negative holdout results. This choice loses a possible learning signal but avoids presenting unstable evidence as a reliable gain.

5. Experiment design

5.1 Final public matrix

Every final comparison uses seeds 101, 102, and 103 across development, sparse, cold start, delayed, shift, and drift. That is 18 complete episodes per method. Each episode has 200 members, 60 decision days, and 40 follow-up days. The same seed and scenario pairs are used for every method.

The public seed set is a reproducibility block, not an untouched statistical holdout. We report all episodes and do not select a favorable subset.

5.2 Methods

Method	Purpose
`adaptive_greedy`	Submitted stable clarification policy.
`greedy`	Required supplied greedy baseline.
`no_asks`	Required no-clarification baseline and clarification ablation.
`random`	Required random-feasible baseline.
`adaptive_input_order_greedy`	Ablation that removes stable clarification ordering.

5.3 Seeds and training

The submitted method has no training seeds and no inference random seed. It is deterministic. Seeds 101 to 103 are declared public comparison seeds.

A rejected research-only outcome model used generated seeds 2001 to 2040 for training. Its asset is not copied into the Docker image and does not affect the submitted policy. The permanent seed ledger is `experiments/seed_registry.json`.

5.4 Uncertainty

The machine-readable summary compares paired episodes and resamples complete seed groups. Each resampled seed keeps all six scenario variants together. This preserves the shared generated world and gives each scenario family equal weight.

Three public seed groups cannot support a confident performance claim. We use the interval to show the uncertainty, not to promote a method. Earlier method selection used separate seed blocks recorded in the decision log.

6. Results

All values below come from the files linked by `results/round2_summary.json`. The primary score is the mean of six scenario means. MSMI is one qualifying pair, not two people.

6.1 Overall public results

Method	Primary MSMI/100	Coverage	Mutual accepts/100	Ask cost	MSMI outcomes
`adaptive_greedy`	0.4444	0.3700	5.2222	206.67	16
`greedy`	0.3889	0.3717	5.5000	206.67	14
`no_asks`	0.1389	0.1428	1.7500	0.00	5
`random`	0.2778	0.3767	5.3333	206.67	10
Input-order ablation	0.3889	0.3717	5.5000	206.67	14

6.2 Submitted method by scenario

Scenario	MSMI/100	Coverage	Mutual accepts/100	Ask cost
Development	0.5000	0.4283	5.8333	200.00
Sparse	0.0000	0.1450	1.3333	196.00
Cold start	0.6667	0.3617	5.3333	244.00
Delayed	0.3333	0.4300	5.8333	200.00
Shift	0.6667	0.4267	7.5000	200.00
Drift	0.5000	0.4283	5.5000	200.00

6.3 Interpretation

The submitted policy records 16 MSMI outcomes, compared with 14 for greedy and 5 for no clarification. The primary-score difference from greedy is +0.0556 MSMI per 100 arrived members. Coverage is 0.0017 lower than greedy.

The gain comes from cold start and shift, while greedy does better in sparse and delayed. With three seed groups, one outcome changes the primary score by about 0.0278. We therefore describe the result as promising public evidence, not a confirmed advantage.

The paired seed-grouped bootstrap difference from greedy is 0.0556, with a 95% interval from 0.0000 to 0.0833. The lower endpoint is zero. This interval does not support a confident claim that the gain will repeat.

The zero sparse-scenario mean is the clearest weakness. Sparse supply leaves few feasible edges, and the current policy has no validated waiting rule or scarcity-aware ask rule that improves this scenario without harming others.

6.4 Service, feedback, and runtime

Across the 18 submitted-policy episodes, the policy made 1,417 assignments and served 1,332 of 3,600 arrived members at least once. The median first-service wait among served members was 5 days. The 90th percentile was 16 days, and the maximum was 50 days. A total of 2,268 members were unserved. The simulator reported 783 missing feedback events, 188 mutual acceptances, and 152 dates.

The local evaluator made 2,160 policy calls. The slowest call took 8.09 seconds, below the 10-second limit. Another workspace was consuming substantial CPU during this run, so aggregate local runtime is not used as a method-comparison claim. The separate container check is the packaging and limit proof.

7. Correctness and edge cases

The repository has tests for empty pools, odd populations, one-edge graphs, conflicting duplicates, future observations, declined answers, day-boundary feedback, repeated pairs, mixed pools, refreshed post-ask state, request replay, hash-seed changes, dense graphs, payload limits, and action determinism. The full register is `docs/EDGE_CASE_REGISTER.md`.

At report freeze, the suite contains 118 tests. The release checks also verify all 71 public data files, JSON finiteness, the 1 MiB limits, and the 10-second per-call boundary.

8. Runtime and packaging

The image pins `python:3.12.7-slim` to digest `sha256:60d9996b6a8a3689d36db740b49f4327be3be09a21122bd02fb8895abb38b50d`. It copies only `kit.py`, `adaptive.py`, and `policy.py`. The container runs as user 65534 with no policy dependency installation. Assessed evaluation disables networking, removes Linux capabilities, uses a read-only filesystem, and applies the published CPU, memory, process, and time limits.

The final pinned-image check produced a 45,193,310-byte image and one valid seed-101 development episode. It made 120 container calls. The slowest call took 1.246 seconds, and the 95th percentile took 0.969 seconds. The episode scored 0.5 MSMI per 100 arrived members. Full evidence is in `results/container_check.json`.

Run the final checks from the repository root:

```
python -m unittest -v
python verify_data.py
python evaluate.py --seeds 101,102,103 --variants all --output results/final.json
python experiments/build_round2_summary.py --output results/round2_summary.json
docker build -t sequential-policy:submission .
python evaluate.py --image sequential-policy:submission --seeds 101 --variants development --output results/
container_check.json
```

The full local matrix uses `--baseline` to select each named comparison. The exact commands are in `README.md`.

9. Provenance and licences

The policy uses the supplied synthetic simulator and public synthetic data. It does not contain real profiles or organiser-only data. The submitted method uses Python standard-library code written for this project. It has no external model, API, service, or training-compute requirement.

The repository includes the starter `LICENSE`, the `DATA_LICENSE.md`, the `LUMINAX_LICENSE.md`, and `THIRD_PARTY_NOTICES.md`.

10. Limitations and failure modes

- The simulator is synthetic. Its outcomes do not establish real dating effectiveness or fairness.
- Three public seeds are too few for a precise final comparison.
- Equal-value soft fields are a coarse signal and are not calibrated.
- Greedy allocation can miss a better global batch.
- The policy does not learn from delayed outcomes.
- Stable clarification order can change who is served when the budget binds.
- Sparse feasible graphs can leave many people unserved.
- Declined hard answers can permanently block an otherwise possible edge.
- The policy reports no causal, demographic-parity, or production-safety claim.

These limits are part of the submission, not future-work claims disguised as completed features.

11. Reproducibility boundary

The final GitHub issue must name one full 40-character commit SHA. That commit contains the policy, Dockerfile, result JSON, report, PDF, licences, and tests. The evaluator can reproduce the public results without credentials. Private assessment remains outside the repository and supplies only observable JSON to the container.